# Packet Wizardry with **pcapsh**

Build, bend and break packets in one line

---

A hands-on tour

libpcapng • `man pcapsh`

# What if packets were just... sentences?

```
IP(dst="8.8.8.8")/UDP(dport=53)/DNS(rd=1,qd=DNSQR(qname="example.com"))
```

# What if packets were just... sentences?

```
IP(dst="8.8.8.8")/UDP(dport=53)/DNS(rd=1,qd=DNSQR(qname="example.com"))
```

- That line is a DNS query. Lengths, checksums and protocol numbers fill themselves in.
- Type it at a prompt, or save it in a file and run it.
- One `wrpcap()` later, Wireshark opens it like a real capture.

# What if packets were just... sentences?

```
IP(dst="8.8.8.8")/UDP(dport=53)/DNS(rd=1,qd=DNSQR(qname="example.com"))
```

- That line is a DNS query. Lengths, checksums and protocol numbers fill themselves in.
- Type it at a prompt, or save it in a file and run it.
- One `wrpcap()` later, Wireshark opens it like a real capture.

**pcapsh in one sentence**

A Scapy-flavoured packet shell, written in C on top of libpcapng, that speaks dozens of protocols and writes pcapng you can open anywhere.

# Why you will like it

**Instant start**
one binary, no Python, no venv

**Scapy syntax**
stack layers with `/`

**Real pcapng out**
Wireshark, tshark, tcpdump

**Scriptable**
loops, variables, `#!` scripts

**TCP that just works**
sessions track seq/ack for you

**Read captures too**
Wireshark-style display filters

**Invent protocols**
five lines, no recompiling

**Friendly REPL**
Tab completion, colours, history

# Your first packets

# Hello, packet

Start the shell and type a layer name:

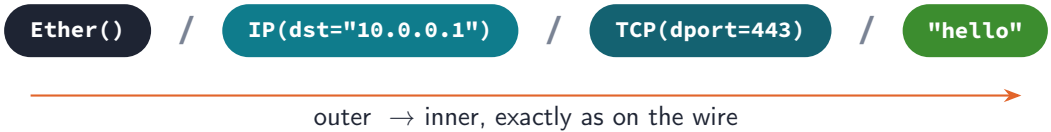
```
$ pcapsh
pcapsh >>> IP()
<IP version=4 ihl=5 tos=0 len=auto id=1 flags=0x0 frag=0 ttl=64
    proto=auto chksum=auto src=127.0.0.1 dst=127.0.0.1>
```

- Every field has a sensible default.
- `auto` means *"I will compute this when it matters"*.
- Override only what you care about:

```
IP(dst="10.0.0.1", ttl=1)
```

Press `Tab` to complete protocol names, functions and variables.

# The slash: stacking layers




Each layer tells the one before it who comes next:

- `Ether.type` becomes 0x0800 because IP follows,
- `IP.proto` becomes 6 because TCP follows,
- `IP.len` and both checksums cover everything after them.

A bare string at the end is a raw payload.

# Look inside: hexdump, raw, ls

```
pcapsh >>> hexdump(IP(dst="10.0.0.1")/ICMP()/"hi!")
0000  45 00 00 1F 00 01 00 00  40 01 F1 DB 7F 00 00 01  E.......@.......
0010  0A 00 00 01 08 00 6E 96  00 00 00 00 68 69 21     ......n.....hi!

pcapsh >>> raw(IP()/UDP())
'E\x00\x00\x1c\x00\x01\x00\x00@\x11|\xce\x7f\x00\x00\x01\x7f\x00\x00\x01\x005\x005...'

pcapsh >>> ls(ARP)
ARP fields:
  htype    uint16   1                  [ETHERNET=0x1]
  op       uint16   1                  [REQUEST=0x1, REPLY=0x2]
  sha      mac      00:00:00:00:00:00
  spa      ip4      0.0.0.0            ...
```

`ls()` with no argument lists every protocol pcapsh knows.

# Speaking the values

## Numbers & maths

```
TCP(dport=0x1BB)
UDP(sport=49000+7)
IP(id=0x100*2+1)
```

## Addresses are strings

```
IP(src="192.0.2.1")
Ether(src="02:00:c0:00:02:01")
```

## Escapes in strings

```
"GET / HTTP/1.0\r\n\r\n"
"\xfeSMB binary \x00 ok"
```

## Flags & names

```
TCP(flags="SA")       # SYN+ACK
TCP(flags=0x12)       # same thing
ARP(op=REPLY)         # enum name
DNSQR(qtype=MX)
```

/ is reserved for stacking, so maths has + - * but no division.

# Variables: name it, reuse it, stack it

```
c = Ether(src="02:00:0a:00:01:01")/IP(src="10.0.1.1",dst="10.0.0.1")
v = Ether(src="02:00:0a:00:00:01")/IP(src="10.0.0.1",dst="10.0.1.1")

wrpcap("chat.pcapng", c/UDP(dport=9999)/"ping")
wrpcap("chat.pcapng", v/UDP(sport=9999)/"pong")
```

- A variable holds a packet, raw bytes, a TCP session or a number.
- A stored packet is a layer: `c/UDP(...)` keeps building on it.
- Define the two endpoints once and every later line stays short.

# Long packet? Keep typing

A statement carries on while a bracket is open or a line ends with `/` or `,`:

```
answer = IP(src="8.8.8.8", dst="192.168.1.10")/
         UDP(sport=53, dport=54321)/
         DNS(id=0xC0DE, qr=1, rd=1, ra=1,
             qd=DNSQR(qname="pcapsh.rocks"),            # the question
             an=DNSRR(rrname="pcapsh.rocks", type=A,    # the answer
                      ttl=300, rdata="10.13.37.1"),
             ancount=1)
```

- Comments on each line are fine; brackets inside strings do not count.
- In the REPL the prompt turns into `...`; an empty line submits.
- Wireshark hex dumps can be pasted over several lines into `fromhex("...")`.

# From shell to Wireshark

```
wrpcap("hello.pcapng", Ether()/IP(dst="10.0.0.1")/ICMP()/"hi!")
wrpcap("hello.pcapng", Ether()/IP(src="10.0.0.1")/ICMP(type=0)/"hi!")
```

- First call creates the file, later calls append.
- Always Ethernet link type: an `Ether()` is added if you skipped it.

```
$ wireshark hello.pcapng          # or: tshark -r hello.pcapng
```

Appending is handy but sticky: `rm` the file before a fresh run.

# Protocols, the fun ones

# DNS without tears

```
q   = IP(src="192.168.1.10",dst="8.8.8.8")/UDP(sport=54321,dport=53)
r   = IP(src="8.8.8.8",dst="192.168.1.10")/UDP(sport=53,dport=54321)
ask = DNSQR(qname="pcapsh.rocks")
ans = DNSRR(rrname="pcapsh.rocks",type=A,ttl=300,rdata="10.13.37.1")

wrpcap("dns.pcapng", q/DNS(id=0xC0DE,rd=1,qd=ask))
wrpcap("dns.pcapng", r/DNS(id=0xC0DE,qr=1,rd=1,ra=1,qd=ask,an=ans,ancount=1))
```

```
$ tshark -r dns.pcapng
 1  192.168.1.10 → 8.8.8.8  DNS  Standard query 0xc0de A pcapsh.rocks
 2  8.8.8.8 → 192.168.1.10  DNS  Standard query response 0xc0de ... A 10.13.37.1
```

Passing `qd=` bumps `qdcount` for you; answers need `ancount=1`.

# Who has 10.0.0.1? Tell 10.0.0.100

```
wrpcap("arp.pcapng", Ether()/ARP(op=REQUEST, spa="10.0.0.100", tpa="10.0.0.1"))
wrpcap("arp.pcapng", Ether()/ARP(op=REPLY, sha="aa:bb:cc:dd:ee:01",
                                 spa="10.0.0.1", tpa="10.0.0.100"))
```

`Ether.type` follows the next layer: 0x0806 here, 0x0800 before IP.

## TCP options are fields too:

```
IP(dst="10.0.0.1")/TCP(dport=443,flags="S",mss=1460,sack_perm=1)
```

```
40000 → 443 [SYN] Seq=0 Win=8192 Len=0 MSS=1460 SACK_PERM
```

# Packet inception: tunnels

```
# a TCP SYN, in Ethernet, in VXLAN, in UDP, in IP
wrpcap("vxlan.pcapng", IP(src="10.0.0.1",dst="10.0.0.2")/
    UDP(sport=49152,dport=4789)/VXLAN()/
    Ether(src="02:aa:00:00:00:01")/IP(src="172.16.0.1",dst="172.16.0.2")/
    TCP(sport=40000,dport=8080,flags="S"))

# GRE: IPv4 carried over IPv4
wrpcap("gre.pcapng", IP(src="10.0.0.1",dst="10.0.0.2")/GRE(proto=IPV4)/
    IP(src="192.168.1.1",dst="192.168.2.1")/TCP(dport=80,flags="S"))
```



# Break it on purpose

Lengths and checksums are computed *unless you set them*. Fuzzers, IDS tests and dissector bugs love this:

```
wrpcap("evil.pcapng", IP(dst="10.0.0.1",ttl=0,chksum=0xdead)/
                      UDP(dport=53,len=4)/"oops")
```

```
$ tshark -r evil.pcapng -o ip.check_checksum:TRUE -V
    [Expert Info (Note/Sequence): "Time To Live" only 0]
    [Expert Info (Error/Checksum): Bad checksum [should be 0x31cb]]
 1  127.0.0.1 → 10.0.0.1  UDP  53 → 53 [BAD UDP LENGTH 4 < 8]
```

Same trick for `IP(len=...)`, `IP(proto=...)`, `TCP(chksum=...)`, `Ether(type=...)`.

# 75+ protocols ship in the box

Native C layers plus `.posa` definitions loaded at start-up; try `ls()` and `ls(NAME)`.

**LAN**
ARP, STP, CDP, LACP, 802.1Q, EAPOL, VRRP, HSRP, PPPoE

**Internet**
DNS, DHCP, NTP, IPv6, GRE, VXLAN, MPLS, BGP, OSPF, SCTP

**Enterprise**
SMB2, DCERPC, LDAP, Kerberos, RADIUS, NBNS, RDP, IKEv2

**IoT & industry**
ModbusTCP, DNP3, BACnet, MQTT, CoAP, OpenFlow, PTPv2, Wake-on-LAN

```
IP()/TCP(dport=445)/NBT()/SMB2(command=READ, tree_id=1)
IP()/UDP(sport=49152,dport=1812)/RADIUS(code=ACCESS_REQUEST, identifier=7)
```

# Scripts: write once, capture forever

# A script is just the lines you typed

```
#!/usr/bin/env pcapsh
# ping.pcapsh -- two echo request/reply pairs
a = IP(src="10.0.0.1",dst="10.0.0.2")
b = IP(src="10.0.0.2",dst="10.0.0.1")
wrpcap("ping.pcapng", a/ICMP(type=8,id=1,seq=1)/"hello")
wrpcap("ping.pcapng", b/ICMP(type=0,id=1,seq=1)/"hello")
wrpcap("ping.pcapng", a/ICMP(type=8,id=1,seq=2)/"hello")
wrpcap("ping.pcapng", b/ICMP(type=0,id=1,seq=2)/"hello")
```

```
$ chmod +x ping.pcapsh && ./ping.pcapsh
$ pcapsh -o lab42.pcapng ping.pcapsh   # same script, different output file
$ pcapsh -e 'hexdump(IP()/ICMP())'      # one-liner from any shell
```

# Loops: a port scan in two lines

```
t = IP(src="10.0.0.66",dst="10.0.0.1")
for $p in range(20, 26):
    wrpcap("sweep.pcapng", t/TCP(sport=40000+$p,dport=$p,flags="S"))
```

| | |
|---|---|
| `range(n)` | 1 ... n |
| `range(a,b)` | a ... b-1 |
| `range(a,b,s)` | a, a+s, ... |

Indented lines form the body.

```
$ pcapsh sweep.pcapsh
$ tshark -r sweep.pcapng
10.0.0.66 → 10.0.0.1  40020 → 20 [SYN]
10.0.0.66 → 10.0.0.1  40021 → 21 [SYN]
10.0.0.66 → 10.0.0.1  40022 → 22 [SYN]
...
10.0.0.66 → 10.0.0.1  40025 → 25 [SYN]
```

`range(n)` counts from 1, like packet numbers. `$p` works in numeric fields only.

# Generate traffic by the bucket

```
# 20 NTP clients hitting one server, from ephemeral ports
cli = IP(src="10.0.1.1",dst="10.0.0.2")
srv = IP(src="10.0.0.2",dst="10.0.1.1")
for $i in range(20):
    wrpcap("ntp.pcapng", cli/UDP(sport=49151+$i,dport=123)/
                         NTP(li_vn_mode=CLIENT))
    wrpcap("ntp.pcapng", srv/UDP(sport=123,dport=49151+$i)/
                         NTP(li_vn_mode=SERVER,stratum=1))

# 50 DNS queries, each with its own ports and ids
q = DNSQR(qname="example.org")
for $i in range(50):
    wrpcap("dns50.pcapng", IP(dst="9.9.9.9",id=$i)/UDP(sport=50000+$i)/
                           DNS(id=$i*7,rd=1,qd=q))
```



Offset your ports (`49151+$i`): `sport=$i` lands on Echo, Discard, Chargen...

# TCP that talks back

# Hand-made TCP is a maths exam

```
TCP(flags="S",  seq=1000)
TCP(flags="SA", seq=5000, ack=1001)
TCP(flags="A",  seq=1001, ack=5001)
TCP(flags="PA", seq=1001, ack=5001)/"GET / HTTP/1.1\r\n\r\n"
TCP(flags="PA", seq=5001, ack=1019)/"HTTP/1.1 200 OK..."   # 1019? 1020? hmm
```

# Hand-made TCP is a maths exam

```
TCP(flags="S",  seq=1000)
TCP(flags="SA", seq=5000, ack=1001)
TCP(flags="A",  seq=1001, ack=5001)
TCP(flags="PA", seq=1001, ack=5001)/"GET / HTTP/1.1\r\n\r\n"
TCP(flags="PA", seq=5001, ack=1019)/"HTTP/1.1 200 OK..."   # 1019? 1020? hmm
```

Let the shell do the counting. ☺

# `TCPSession`: a whole web visit

```
s = TCPSession("192.168.1.100", "93.184.216.34",
               54321, 80)
wrpcap("http.pcapng", syn(s))
wrpcap("http.pcapng", syn_ack(s))
wrpcap("http.pcapng", ack(s))
wrpcap("http.pcapng", client_send(s,
    "GET / HTTP/1.1\r\nHost: example.com\r\n\r\n"))
wrpcap("http.pcapng", server_send(s,
    "HTTP/1.1 200 OK\r\nContent-Length: 3\r\n\r\nhi!"))
wrpcap("http.pcapng", client_fin(s))
wrpcap("http.pcapng", server_fin_ack(s))
```

```
$ tshark -r http.pcapng
1 TCP   54321 → 80 [SYN] Seq=0
2 TCP   80 → 54321 [SYN, ACK] Seq=0 Ack=1
3 TCP   54321 → 80 [ACK] Seq=1 Ack=1
4 HTTP  GET / HTTP/1.1
5 HTTP  HTTP/1.1 200 OK
6 TCP   54321 → 80 [FIN, ACK] Seq=38 Ack=42
7 TCP   80 → 54321 [FIN, ACK] Seq=42 Ack=39
```

Every seq and ack lines up, so Wireshark's *Follow TCP Stream* replays the whole conversation.

# The session cheat sheet

| Function | Direction | Flags |
|---|---|---|
| `syn(s)` | client → server | S |
| `syn_ack(s)` | server → client | SA |
| `ack(s)` / `tcp_ack(s)` | client → server | A |
| `client_send(s, data)` | client → server | PA |
| `server_send(s, data)` | server → client | PA |
| `client_fin(s)` | client → server | FA |
| `server_fin_ack(s)` | server → client | FA |

- Each call returns a full Ether/IP/TCP packet and advances seq/ack.
- MACs derive from IPs, so endpoints are stable across a capture.
- Run several sessions at once; they interleave in write order.
- `data` may be bytes too: `client_send(s, raw(SMB2()))`.

# TLS, minus the secrets

Handshake records ready to drop after TCP; NULL cipher, so Wireshark sees everything.

```
c = IP(src="10.0.1.1",dst="10.0.0.1")
v = IP(src="10.0.0.1",dst="10.0.1.1")
wrpcap("tls.pcapng", c/TCP(sport=50000,dport=636,flags="PA",seq=1,ack=1)/
                     TLS_CLIENT_HELLO())
wrpcap("tls.pcapng", v/TCP(sport=636,dport=50000,flags="PA",seq=1,ack=51)/
                     TLS_SERVER_HELLO())
wrpcap("tls.pcapng", c/TCP(sport=50000,dport=636,flags="PA",seq=51,ack=48)/
                     TLS()/LDAP(op_tag=BIND_REQUEST))
```

```
1  10.0.1.1 → 10.0.0.1  TLSv1.2  Client Hello
2  10.0.0.1 → 10.0.1.1  TLSv1.2  Server Hello
3  10.0.1.1 → 10.0.0.1  TLSv1.2  Application Data
```

`TLS()` wraps whatever follows. Record sizes: Hello 50/47, Certificate 12, CCS 6, Finished 21.

# Reading, dissecting, editing

# rdpcap: tshark-lite with display filters

```
pcapsh >>> rdpcap("sweep.pcapng", "tcp.dstport in {22..24}")
  3  ...  TCP  10.0.0.66 → 10.0.0.1  40022 → 22 [SYN] Seq=3735928320 ...
  4  ...  TCP  10.0.0.66 → 10.0.0.1  40023 → 23 [SYN] Seq=3735928320 ...
  5  ...  TCP  10.0.0.66 → 10.0.0.1  40024 → 24 [SYN] Seq=3735928320 ...
3 / 6 packet(s) matched
```

```
"dns"
"ip.addr == 192.168.0.0/16"
"tcp.port in {80, 443}"
"dns && !(ip.src == 8.8.8.8)"
```

```
"udp.dstport >= 1024 && udp"
"eth.src[0:3] == 02:00:c0"
"http.request"
"tcp.flags.syn == 1"
```

Reads pcap and pcapng. `-k keylog` (or `$SSLKEYLOGFILE`) loads TLS keys.

# Paste hex, get meaning: `fromhex` + `show`

Copy bytes from Wireshark (*Copy → as Hex Dump*), paste, name the stack:

```
pcapsh >>> d = fromhex("45 00 00 34 00 01 00 00 40 11 f6 c4 c0 a8 01 05
                       08 08 08 08 c3 a8 00 35 00 20 41 a1 12 34 01 00
                       00 01 00 00 00 00 00 00")
pcapsh >>> show("IP/UDP/DNS", d)
<IP src=192.168.1.5 dst=8.8.8.8 ttl=64 proto=17(UDP) len=52 |
<UDP sport=50088 dport=53 len=32 |
<DNS id=4660 flags=0x0100 qdcount=1 ancount=0 nscount=0 arcount=0 |
```

- `fromhex` eats plain streams, spaced bytes, or full Wireshark dumps with offsets and ASCII.
- `show` walks the stack; IP and TCP options are skipped by their length fields.

# Packet surgery on an existing capture

```
# dissect the first three packets
for $n in range(3):
    show("Ether/IP/TCP", frompcapng("sweep.pcapng", $n))

# swap packet 3 for a RST, in place
replacepkt("sweep.pcapng", 3,
           IP(src="10.0.0.66",dst="10.0.0.1")/TCP(sport=40022,dport=22,flags="R"))
```

```
Replaced packet #3 in sweep.pcapng (54 bytes)
pcapsh >>> rdpcap("sweep.pcapng")
  2  ...  TCP  10.0.0.66 → 10.0.0.1  40021 → 21 [SYN] Seq=3735928320 ...
  3  ...  TCP  10.0.0.66 → 10.0.0.1  40022 → 22 [RST] Seq=0 ...
  4  ...  TCP  10.0.0.66 → 10.0.0.1  40023 → 23 [SYN] Seq=3735928320 ...
```

`frompcapng(file, n)` returns raw bytes: feed them to `show`, `hexdump` or `wrpcap`.

# Invent your own protocol

# The Pizza Ordering Protocol™

```
protocol Pizza
    uint8 size defaults(2)
        SMALL  = 1
        MEDIUM = 2
        LARGE  = 3
    uint8 topping defaults(1)
        CHEESE    = 1
        PEPPERONI = 2
        PINEAPPLE = 3
    uint16 quantity defaults(1)
    cstring note defaults("extra hot")
end
```

```
p = IP(dst="10.0.0.42")/UDP(sport=50000,dport=8080)/
    Pizza(size=LARGE, topping=PINEAPPLE,
          quantity=2)
hexdump(p)
```

```
0010  0A 00 00 2A C3 50 1F 90  00 16 79 36 03 03 00 02
0020  65 78 74 72 61 20 68 6F  74 00                    extra hot.
```

Usable the moment end is read: build it, dump it, `ls(Pizza)` it, and dissect it:

```
pcapsh >>> show("Pizza", fromhex("03 03 00 02 6e 6f 20 61 6e 63 68 6f 76 69 65 73 00"))
<Pizza size=LARGE(3) topping=PINEAPPLE(3) quantity=2 note='no anchovies' |
```

# Keep it in a `.posa` file

```
# game.posa
Object<GAME> GAME_HELLO
    uint8 opcode defaults(1)
        HELLO = 1
    cstring player defaults("neo")

Object<GAME> GAME_SCORE
    uint8 opcode defaults(2)
        SCORE = 2
    uint32 points defaults(0)
```

`Object<GAME>` groups messages: `show("GAME", ...)` picks the member whose opcode matches.

```
$ pcapsh game.posa
>>> show("GAME", fromhex("02 00 00 05 39"))
<GAME_SCORE opcode=SCORE(2) points=1337 |
>>> show("GAME", fromhex("01 74 72 69 6e 69 74 79 00"))
<GAME_HELLO opcode=HELLO(1) player='trinity' |
>>> hexdump(GAME_HELLO(player="morpheus"))
0000  01 6D 6F 72 70 68 65 75  73 00     .morpheus.
```

Load with `-p`, `load("game.posa")`, or forever via `~/.pcapsh_protos.posa`.

# The posa toolbox

| Type | What it is |
|---|---|
| `uint8 uint16 uint32 uint64` | big-endian integers |
| `le_uint16 le_uint32 le_uint64` | little-endian (hello, SMB2) |
| `mac`, `ip4` | addresses: `defaults(10.0.0.1)` |
| `bytes<N>`, `str<N>` | fixed width, zero padded |
| `cstring` | NUL-terminated text |
| `string x until "\r\n"` | text up to a delimiter (HTTP, SIP…) |
| `enum<uint8>` | integer with names |

Indented `NAME = value` lines under a field give it enum names.

Full language reference: `doc/posa.md`.

# Wrapping up

# Coming from Scapy? You already know this

| Task | Scapy | pcapsh |
|---|---|---|
| Build | `IP()/TCP()` | `IP()/TCP()` |
| Inspect | `hexdump(p)`, `ls(TCP)` | `hexdump(p)`, `ls(TCP)` |
| Save | `wrpcap("f", p)` | `wrpcap("f", p)` *(appends)* |
| Read | `rdpcap("f")` | `rdpcap("f", "filter")` |
| Loop | `for i in range(10):` | `for $i in range(10):` *(1..10)* |
| New proto | a Python class | 5 lines of `protocol...end` |
| TCP state | do it yourself | `TCPSession` |
| Multi-line | Python rules | open bracket or trailing `/` |
| Start-up | Python + imports | one C binary |

pcapsh builds and writes files; it does not send packets on the wire.

# Your turn: five challenges

- **Ping pong.** Ten ICMP echo request/reply pairs with `seq=1..10`, in one loop.
- **DNS liar.** A query for `bank.example` and an answer pointing it to `10.66.6.6`.
- **Knock knock.** SYNs to ports 7000, 8000, 9000, then a full `TCPSession` to port 22.
- **Syslog storm.** 100 RFC 3164 messages: `UDP(dport=514)/"<38>Oct 24 10:00:01 host app: hi"`.
- **Pizza over TCP.** Order three pizzas through `client_send(s, raw(Pizza(...)))` and read them back with `show`.

Check every answer with `tshark -r` or `rdpcap`.

# Pocket reference

### Run

```
pcapsh                          REPL
pcapsh file.pcapsh              script
pcapsh -e 'EXPR'                one-shot
pcapsh -p x.posa                extra protocols
pcapsh -o out.pcapng s.pcapsh
pcapsh --version
```

### Functions

```
hexdump(p)  raw(p)  ls([P])
wrpcap(file, p)
rdpcap(file [, filter])
frompcapng(file, n)
replacepkt(file, n, p)
fromhex("..")  show("A/B", bytes)
load("x.posa")  RandShort()
```

**Build** `A()/B(f=v)/"payload"`

`man pcapsh` and `help()` inside the shell

Now go make some packets.

```
$ pcapsh
```